Learning Cuda Notes

__global__ function
- means function runs on gpu
- called a kernel
- launched from cpu
- runs many of them in parallel

threadIdx.x
- ID of thread

launching kernel
Global function <<< # of blocks, # of threads per block >>>();

CudaDeviceSynchronize
- When launching kernel
  - GPU will run asynchronously
    - Out of sync with cpu
- CPU has synchronize and waits for gpu to finish
- Don't otherwise, kernels will start in order if chained

Parallelizing Work to Run on GPUs *

Warp Divergence
- When threads in the same warp disagree on conditions
  - Has to run both paths one at a time now
    - Messes with parallelism
- In general
  - Avoid branching a lot in kernels
  - Try to make all threads agree
  - Edge Case branching is fine
  - Don't make branch condition on threadIdx such that it splits the warp
  - Branching on BlockIdx is fine since it affects the all warps in a block

Memory Coalescing
- Threads in the same warp should pull from consecutive memory addresses
  - Sort of like cache locality

Global Memory
- Every warp in block has to pull from same memory

Warp 0 →
⋮ → pulls same global memory
Warp n →

inefficient

- Keep one time data here

Shared mem
- shared per block!
- Block loads once into shared memory

Warp 0 →
⋮ → pulls from shared (faster)
Warp n →

- Keep reused data here

Hierarchy - fast to slow
- try to optimize for this
- Registers → single values used by 1 thread
- Shared mem → data reused in a block
- L1/L2 cache → automatic, local data
- Global memory → data only used once

Streaming Multiprocessor (SM)
- Contains
  - Cuda Cores (ALUs)
  - Warp Schedulers
  - Registers
  - Shared memory
  - Caches
  - Tensor Cores
- Assigned blocks to work on